

Quantization for Reducing the Size of Large Language Models

Prepared by Mazen Mahmoud

September 24, 2025

Abstract

Transformer-based models such as BERT and LLaMA achieve state-of-the-art performance in natural language processing but are often extremely large and costly to deploy. **Quantization** is a practical technique to reduce model size and speed up inference with minimal accuracy loss. This report introduces quantization, compares main approaches, provides code examples, and visualizes its impact on memory.

1 Introduction

Large Language Models (LLMs) such as BERT [1] and LLaMA [2] can have hundreds of millions or billions of parameters. For instance, BERT-base has 110M parameters ($\sim 420\text{MB}$ in FP32) and LLaMA-7B needs about 13GB. Such models are hard to run on laptops, edge devices, or even standard GPUs.

2 Quantization: The Core Idea

Quantization converts 32-bit floating point (FP32) weights into lower-precision formats (e.g., INT8, INT4).

- FP32: 4 bytes per weight
- FP16: 2 bytes per weight (2x smaller)
- INT8: 1 byte per weight (4x smaller)
- INT4: 0.5 byte per weight (8x smaller)

Example: BERT-base (110M params)

- FP32: $\sim 420\text{MB}$
- INT8: $\sim 110\text{MB}$
- INT4: $\sim 55\text{MB}$

3 Techniques

3.1 Post-Training Quantization (PTQ)

Convert a trained FP32 model directly to INT8/INT4.

- Fast, no retraining required
- Slight accuracy drop may occur

3.2 Quantization-Aware Training (QAT)

Simulate quantization during training so the model learns to handle low-precision.

- Better accuracy retention than PTQ
- Requires additional training

4 Code Examples

INT8 Quantization with Hugging Face Transformers

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    load_in_8bit=True,          # Load weights in INT8
    device_map="auto"
)

inputs = tokenizer("Quantization makes models smaller.", return_tensors="pt")
with torch.no_grad():
    outputs = model(**inputs)
print(outputs.logits.shape)
```

4-bit Quantization with QLoRA

```
from transformers import AutoModelForCausalLM
from peft import prepare_model_for_kbit_training

model_name = "meta-llama/Llama-2-7b-hf"

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    load_in_4bit=True,
    device_map="auto"
)
model = prepare_model_for_kbit_training(model)
```

5 Visualization

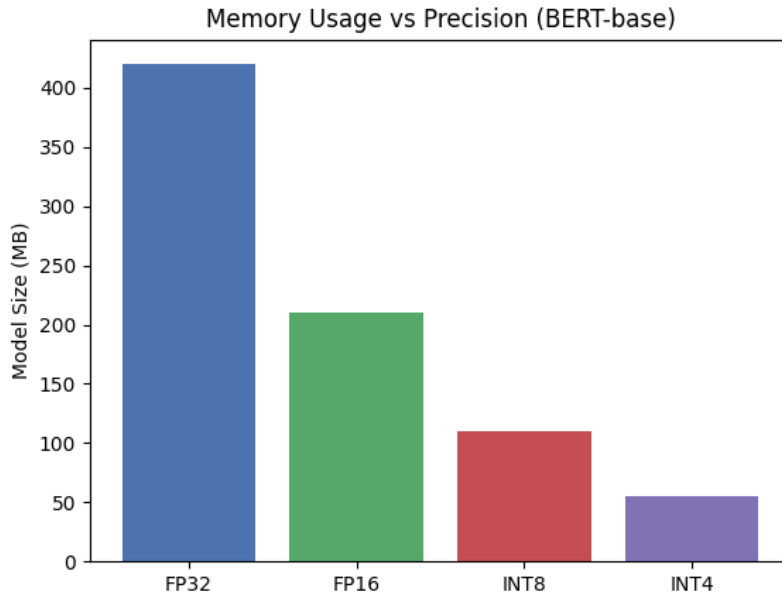


Figure 1: Memory usage of a 110M-parameter model under different precisions.

6 Industry Applications

- **Edge and Mobile Deployment:** Chatbots and summarization running on phones.
- **Cloud Inference:** Cheaper serving by reducing GPU memory per instance.
- **Fine-Tuning:** QLoRA enables adapting very large LLMs on a single consumer GPU.

7 Conclusion

Quantization reduces storage and compute costs of LLMs while keeping most of their accuracy. PTQ works well for quick deployment, while QLoRA and QAT provide higher accuracy when retraining is possible.

References

- [1] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*.
- [2] Touvron, H. et al. (2023). LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971*.
- [3] Dettmers, T., Lewis, M., Shleifer, S., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *arXiv preprint arXiv:2305.14314*.